Experiment:- 12

Problem Statement:-

Manual execution of commands in Linux can be repetitive and inefficient. Bash shell scripting enables users to automate tasks, accept user input, perform arithmetic operations, make decisions using conditional statements, and execute repetitive tasks using loops. These capabilities are essential for system administration, automation, and cyber security workflows.

The problem is to write and execute Bash shell scripts that demonstrate interactive input, positional parameters, arithmetic operations, conditional statements, and loop constructs, thereby automating basic computational and logical tasks.

Aim:- Shell programming using bash: Interactive script, Parameters, Arithmetic operations, Conditional Statement and loops.

Procedure:-

A. Interactive Shell Script

1. Create a script file: nano interactive.sh

```
┌──(srishti_m㉿kali)-[~]
└─$ nano interactive.sh
```

2. Enter the following code: #!/bin/bash echo "Enter your name:" read name echo "Welcome $name" .Save and exit.

```
File  Actions  Edit  View  Help
  GNU nano 8.4                    interactive.sh
#!/bin/bash
echo "enter your name:"
read name
echo "welcome $name"
```

3. Make the script executable: chmod +x interactive.sh

```
┌──(srishti_m㉿kali)-[~]
└─$ chmod +x interactive.sh
```

4. Execute the script: ./interactive.sh

```
┌──(srishti_m㉿kali)-[~]
└─$ ./interactive.sh
enter your name:
srishti mishra
welcome srishti mishra

┌──(srishti_m㉿kali)-[~]
└─$
```

B. Script Using Positional Parameters

5. Create another script: nano params.sh

```
┌──(srishti_m㉿kali)-[~]
└─$ nano params.sh
```

6. Enter the following code: #!/bin/bash echo "Script Name: $0" echo "First Argument: $1" echo "Second Argument: $2"

```
  GNU nano 8.4                    params.shy
#!/bin/bash
echo "script name:$0"
echo "first argement:$1"
echo "secong agrement:$2"
```

7. Execute: ./params.sh Linux Bash

```
┌──(srishti_m㉿kali)-[~]
└─$ ./params.shy linux bash
bash: ./params.shy: Permission denied
```

C. Arithmetic Operations

8. Create script: nano arithmetic.sh

```
┌──(srishti_m㉿kali)-[~]
└─$ nano arithematic.sh

┌──(srishti_m㉿kali)-[~]
└─$
```

9. Enter: #!/bin/bash a=10 b=5 sum=$((a + b)) echo "Sum = $sum"

```
  GNU nano 8.4                    arithematic.sh *
#!/bin/bash
a=10
b=5
sum=$((a+b))
echo"sum = $sum"
```

D. Conditional Statement

10. Create script: nano condition.sh

```
┌──(srishti_m㉿kali)-[~]
└─$ nano condition.sh
```

11. . Enter: 14. #!/bin/bash . echo "Enter a number:" read num . if [$num -gt 0] then . else fi

```
GNU nano 8.4                    condition.shy
#!/bin/bash
echo "enter a number"
read num
if[  $num -gt 0]
then
   echo "positive number"
else
   echo "zero or negative number"
fi
```

E. Loop Constructs

12. Create for-loop script: nano loop.sh

```
┌──(srishti_m㉿kali)-[~]
└─$ nano loop.sh
```

13. Enter: #!/bin/bash . for i in 1 2 3 4 5 . do . done echo "Iteration $i"

```
GNU nano 8.4                    loop.sh *
#!/bin/bash
for i in 1 2 3 4 5
do
  echo "iteration $i"
done
```

Expected Output

- Scripts accept user input and display results correctly.
- Positional parameters are accessed successfully.
- Arithmetic operations produce correct output.
- Conditional statements and loops execute as expected.

Result

Bash shell scripts demonstrating interactive input, positional parameters, arithmetic operations, conditional statements, and loops were successfully created and executed.